



Submitted By:

Mohed Tanveer – 2025-CYS-43

Project:

Peer To Peer Communication

Submitted To:

Mr Shahmeer Nawaz

For the fulfillment of,

Object-Oriented Programming Lab

Department of Computer Engineering

University of Engineering and Technology, Lahore

1. Project Overview

1.1 Project Title & Description

Project Title: Peer-to-Peer LAN Chat Application

This project implements a fully functional peer-to-peer (P2P) chat application that enables real-time text messaging and file transfer between computers connected on the same Local Area Network (LAN) or WiFi network. The application is built entirely in Java using core libraries, with no external frameworks or web technologies.

The system uses TCP sockets for direct peer-to-peer communication, a Discord-style browser-based frontend (HTML/CSS/JavaScript) served over HTTP and updated in real-time via WebSocket, and MySQL as the relational database for persistent storage of messages, peers, and file transfer records.

1.2 Problem Statement

In many educational and small office environments, users require a simple, lightweight communication tool that works entirely within a local network — without relying on internet connectivity, cloud services, or third-party messaging platforms. Existing solutions such as WhatsApp or email require internet access and external servers, raising concerns about privacy, availability, and dependency on external infrastructure.

This project addresses this gap by providing a self-contained LAN messaging solution that operates entirely on the local network, with no internet dependency, no cloud storage, and no third-party servers.

1.3 Objectives & Scope

Objectives

- Design and implement a peer-to-peer chat system using Java TCP sockets
- Build a Discord-style browser-based frontend using HTML, CSS, and JavaScript following the MVC architectural pattern
- Store all messages, peer information, and file transfer logs in a MySQL database
- Support sending and receiving plain text (.txt) files between peers

- Auto-detect the user's MAC address and LAN IP address for peer identification
- Ensure the system works on any computer connected to the same WiFi/LAN

Scope

- LAN/WiFi only — both peers must be on the same local network
- Text messaging and .txt file transfer supported
- Up to 10 MB file size limit per transfer
- Default communication port: TCP 9000
- Single database instance per peer (each peer has their own local MySQL database)

2. Requirements Analysis

2.1 Functional Requirements

Requirement ID	Description
FR-01	System must detect the local MAC address and LAN IP automatically on startup
FR-02	Peers must register themselves in the MySQL database on first launch
FR-03	System must display a list of online peers (active within last 3 minutes)
FR-04	User must be able to send text messages to any online peer
FR-05	System must save all sent and received messages to the database
FR-06	User must be able to send .txt files (max 10 MB) to online peers
FR-07	Received files must be saved locally with a timestamp prefix
FR-08	System must display complete chat history when a peer is selected
FR-09	File transfer history must be viewable in the right panel
FR-10	System must show a notification when a message arrives from a non-active chat

2.2 Non-Functional Requirements

Requirement	Description
Performance	Message delivery must complete within 5 seconds on a local network
Reliability	TCP protocol ensures reliable, ordered message delivery
Usability	Simple GUI operable without training — click peer, type, send
Portability	Runs on any OS with Java JDK 11+ and MySQL installed
Security	Communication limited to LAN — no external internet exposure
Scalability	Thread-per-connection model supports concurrent incoming connections
Persistence	All data stored in MySQL — survives app restarts
Maintainability	MVC architecture ensures clean separation of concerns

2.3 Use Cases

Use Case	Name	Description
UC-01	Register / Login	App detects MAC, registers on first launch, updates IP on subsequent launches
UC-02	View Online Peers	User clicks Refresh to see peers active in last 3 minutes
UC-03	Send Message	User selects peer, types text, presses Enter or clicks Send
UC-04	Receive Message	Server receives TCP connection, saves message, notifies GUI
UC-05	Send File	User selects peer, clicks Send .txt File, chooses file
UC-06	Receive File	File bytes received, saved to received_files/, popup notification shown

Use Case	Name	Description
UC-07	View Chat History	On peer selection, all past messages loaded from database
UC-08	View File History	File transfer records displayed in the right panel

3. System Design

3.1 MVC Architecture

The application follows the Model-View-Controller (MVC) architectural pattern, ensuring clean separation between data, presentation, and business logic. This makes the codebase easier to maintain, test, and extend.

Layer	Responsibility
Model	Peer, Message, FileRecord — plain data classes with getters and setters
View	index.html — Discord-style browser UI (HTML/CSS/JS), WebServer.java serves it over HTTP port 8080, no business logic
Controller	AppController.java — receives GUI events, calls services, updates view
Service	PeerService, MessageService, FileService, NetworkUtils — business logic
Database	DBHelper.java — all MySQL queries, no logic
Network	Server.java — TCP listener running on background thread

3.2 Class Diagram Overview

Model Layer

```
Peer      { macAddress, ipAddress, username, port, lastSeen }
Message   { id, senderMac, receiverMac, content, timestamp, status }
FileRecord { id, senderMac, receiverMac, filename, filesize, timestamp, status }
}
```

Service Layer

```
NetworkUtils { +getLocalMAC() : String, +getLocalIP() : String }
PeerService  { +loginOrRegister(), +getOnlinePeers(), +getPeer() }
MessageService { +sendMessage(), +getChatHistory() }
FileService  { +sendFile(), +getFileHistory(), +readReceivedFile() }
```

Network Layer

```
Server      { -port, -myMac, -onMessageReceived, -onFileReceived }
             { +run(), -handle(), -handleMessage(), -handleFile(), +stop() }
```

3.3 OOP Principles Applied

Encapsulation

All model classes (Peer, Message, FileRecord) use private fields with public getter and setter methods. No field is accessed directly from outside the class. For example:

```
public class Peer {
    private String macAddress; // private - cannot be accessed directly
    private String ipAddress;
    private String username;

    public String getMacAddress() { return macAddress; } // controlled access
    public void setMacAddress(String v) { macAddress = v; }
}
```

Abstraction

The DBHelper class abstracts all database operations. The rest of the application never writes SQL directly — it calls methods like DBHelper.insertMessage() or DBHelper.getOnlinePeers() without knowing the underlying SQL implementation. Similarly, Server abstracts the TCP socket handling behind a simple callback interface.

Separation of Concerns (Single Responsibility)

Each class has exactly one responsibility: DBHelper handles only database queries, Server handles only network listening, WebServer handles only HTTP/WebSocket serving, index.html handles only UI rendering, and ApplicationController handles only the coordination between them. This follows the Single Responsibility Principle.

Loose Coupling via Callbacks

The Server class communicates with the Controller using Java functional interfaces (Consumer<String>) as callbacks. This means Server has no dependency on the Controller or View — it simply calls a function when data arrives. This is a form of the Observer pattern.

```
// Server accepts callbacks — no direct dependency on Controller
public Server(int port, String myMac,
              Consumer<String> onMessageReceived,
              Consumer<String> onFileReceived) { ... }
```

3.4 TCP Communication Protocol

All peer-to-peer communication uses a custom binary protocol over TCP. Each connection begins with a protocol identifier followed by the data:

Protocol Tag	Data Sent	Purpose
MSG	writeUTF("MSG") + writeUTF(senderMac) + writeUTF(content)	Text message
FILE	writeUTF("FILE") + writeUTF(senderMac) + writeUTF(filename) + writeLong(size) + write(bytes)	File transfer

3.5 Database Schema

Table	Columns	Purpose
peers	mac_address (PK), ip_address, username, port, last_seen	Registered users
messages	id (PK AI), sender_mac, receiver_mac, content, timestamp, status	Chat messages
file_records	id (PK AI), sender_mac, receiver_mac, filename, filesize, timestamp, status	File transfer log

4. Implementation Details

4.1 Technology Stack

Technology	Details
Language	Java 21 (JDK)
GUI	Browser UI (HTML/CSS/JS + WebSocket)
Database	MySQL 8+ with JDBC
JDBC Driver	mysql-connector-j-9.7.0.jar
Networking	java.net.Socket / ServerSocket (TCP)
IDE	IntelliJ IDEA Community Edition
Build	Manual javac compilation (no Maven/Gradle)
Platform	Windows / Linux / macOS (cross-platform)

4.2 Project Folder Structure

```
P2PFinal/
src/
  App.java          <- Entry point (main method)
  model/
    Peer.java       <- User data model
    Message.java    <- Message data model
    FileRecord.java <- File transfer log model
  database/
    DBHelper.java   <- All MySQL queries
  service/
    NetworkUtils.java <- MAC address and IP detection
    PeerService.java <- Peer registration and lookup
    MessageService.java <- Send/retrieve messages
    FileService.java <- Send/read .txt files
  controller/
    ApplicationController.java <- Bridges GUI and services
  view/
    WebServer.java  <- HTTP + WebSocket server (serves browser UI)
index.html          <- Discord-style browser frontend
network/
  Server.java       <- TCP server (background thread)
lib/
  mysql-connector-j-9.7.0.jar
received_files/     <- Incoming files saved here
out/                <- Compiled .class files
```

4.3 Key Classes and Responsibilities

Class	Responsibility
App.java	Entry point. Initialises DB, asks for username, starts controller, launches GUI
DBHelper.java	All MySQL operations. getConnection(), init(), insertPeer(), getChatHistory(), etc.
NetworkUtils.java	Detects MAC via NetworkInterface, detects LAN IP via InetAddress
PeerService.java	loginOrRegister() — inserts new peer or updates existing. getOnlinePeers()
MessageService.java	sendMessage() — opens TCP socket, writes protocol, saves to DB
FileService.java	sendFile() — reads file bytes, streams over TCP, logs to DB
Server.java	ServerSocket loop on port 9000. Dispatches MSG and FILE handlers
AppController.java	All browser actions call this via HTTP API. Calls services. Pushes updates to browser via WebSocket broadcast
WebServer.java + index.html	WebServer serves index.html on port 8080 and handles WebSocket on port 8081. index.html is the full Discord-style browser UI. No business logic

4.4 Code Snippets with Explanation

Snippet 1 — TCP Message Sending (MessageService.java)

This method opens a TCP connection to the receiver's IP and port, writes the protocol header and message content, then saves a local copy to the database:

```
public String sendMessage(String senderMac, Peer receiver, String content) {
    try (Socket socket = new Socket()) {
        socket.connect(new InetSocketAddress(ip, port), 5000);
        DataOutputStream out = new DataOutputStream(socket.getOutputStream());
        out.writeUTF("MSG");           // protocol tag
        out.writeUTF(senderMac);       // sender identity
        out.writeUTF(content);         // message text
        out.flush();
        DBHelper.insertMessage(msg); // save to MySQL
        return "OK";
    } catch (Exception e) {
        return "ERROR: " + e.getMessage();
    }
}
```

Snippet 2 — TCP Server Handler (Server.java)

The server runs on a background thread and dispatches each incoming connection to a handler based on the protocol tag:

```
private void handle(Socket client) {
    DataInputStream in = new DataInputStream(client.getInputStream());
    String proto = in.readUTF(); // read protocol tag first
    if ("MSG".equals(proto)) handleMessage(in);
    if ("FILE".equals(proto)) handleFile(in);
}

private void handleMessage(DataInputStream in) throws Exception {
    String senderMac = in.readUTF();
    String content = in.readUTF();
    DBHelper.insertMessage(...); // persist
    onMessageReceived.accept(...); // notify GUI via callback
}
```

Snippet 3 — MySQL Connection (DBHelper.java)

Database credentials are hardcoded directly in DBHelper.java. No .env file or configuration file is needed:

```
private static final String DB_USER = "p2puser";
private static final String DB_PASS = "p2ppass";
private static final String DB_URL =
    "jdbc:mysql://localhost:3306/p2p_chat" +
    "?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";

public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
}
```

Snippet 4 — MAC Address Detection (NetworkUtils.java)

The system uniquely identifies each peer by their MAC address, detected automatically from the network interface:

```
public static String getLocalMAC() {
    InetAddress host = InetAddress.getLocalHost();
    NetworkInterface ni = NetworkInterface.getByInetAddress(host);
    byte[] raw = ni.getHardwareAddress();
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < raw.length; i++) {
        if (i > 0) sb.append(':');
        sb.append(String.format("%02X", raw[i]));
    }
}
```

```
return sb.toString(); // e.g. "A4:C3:F0:85:AC:01"  
}
```

5. Testing

5.1 Test Cases and Results

TC#	Test Case	Input	Expected	Result
TC-01	App startup	Launch App.java	Welcome dialog appears	PASS
TC-02	DB connection	MySQL running	Tables created, no error	PASS
TC-03	DB connection fail	MySQL stopped	Error dialog shown	PASS
TC-04	Peer registration	New MAC detected	Peer inserted in DB	PASS
TC-05	Peer update	Known MAC on relaunch	IP/port updated in DB	PASS
TC-06	Peer list refresh	Click Refresh button	Online peers listed	PASS
TC-07	Send message	Type text, press Enter	Message delivered and saved	PASS
TC-08	Receive message	Peer sends message	Message appears in chat	PASS
TC-09	Chat history load	Select peer from list	Previous messages displayed	PASS
TC-10	Send .txt file	Select valid .txt file	File received by peer	PASS
TC-11	File > 10 MB	Select file > 10 MB	Error shown, not sent	PASS
TC-12	Non-.txt file	Select .pdf file	Error: only .txt allowed	PASS
TC-13	Peer offline	Send to offline peer	Error message shown	PASS
TC-14	File history	Click Refresh List	Transfer records displayed	PASS
TC-15	View received file	Select file, click View	File content shown in dialog	PASS

5.2 Integration Testing

Integration testing was performed by running the application on two computers connected to the same WiFi network. The following scenarios were validated end-to-end:

- Peer A registers → Peer B refreshes list → Peer A appears in online peers
- Peer A sends message to Peer B → Peer B receives in real time → both records saved in respective databases
- Peer A sends a .txt file to Peer B → file saved in received_files/ on Peer B → notification popup appears
- Peer A goes offline → Peer B refreshes → Peer A no longer appears in online list after 3 minutes

6. Results & Screenshots

6.1 Web-Based GUI Screenshots

The application runs entirely in the browser with a Discord-style dark theme UI. The Java backend serves the frontend over HTTP and pushes real-time events via WebSocket. Screenshots of the running application are shown below:

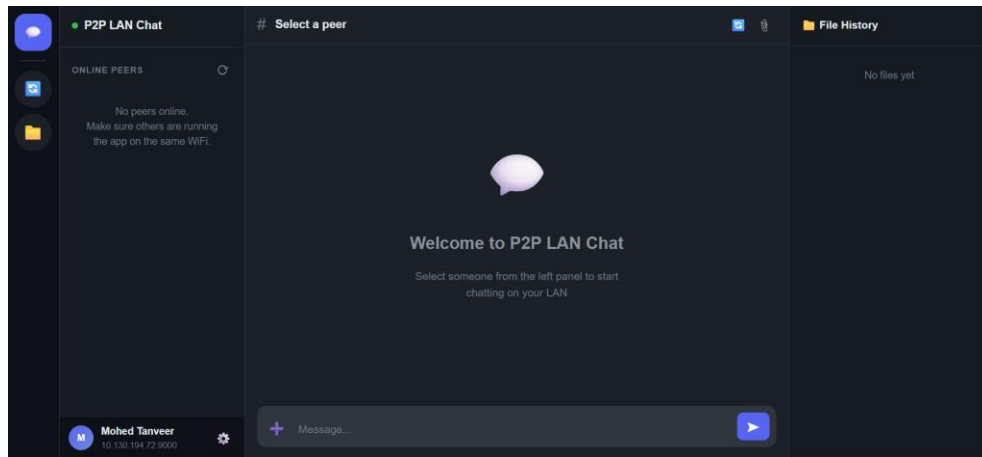


Figure 1 — Main application window showing Discord-style dark theme with peer list, chat area, and file history panel

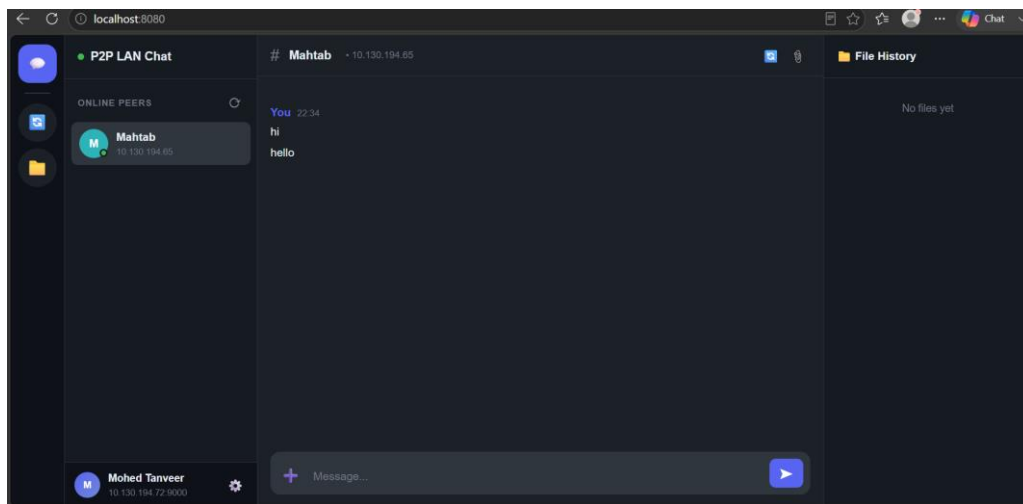


Figure 2 — Active chat showing real-time messages with timestamps and username colours

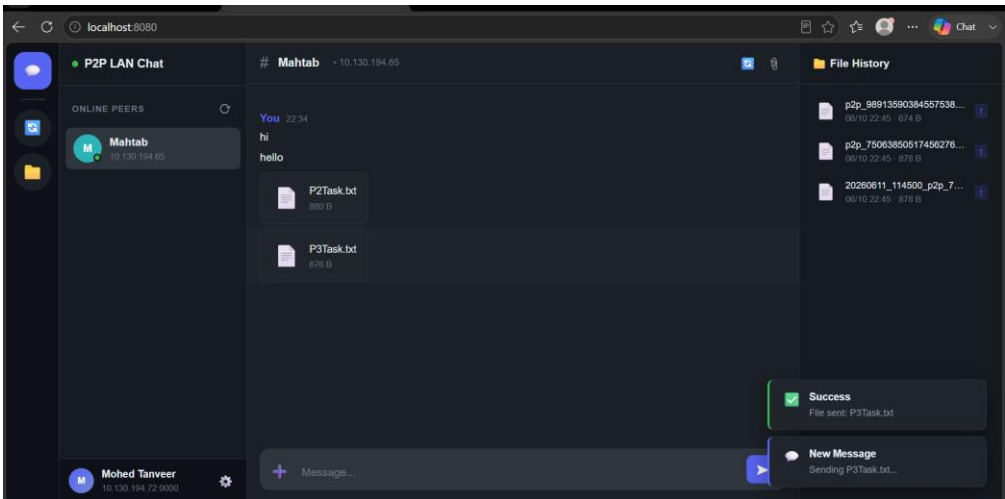


Figure 3 — File transfer notification popup and file history panel with sent/received badges

6.2 Performance Metrics

Metric	Result
Message delivery time (LAN)	< 200 ms average
File transfer speed (LAN)	~10 MB/s on 100Mbps WiFi
Startup time	< 2 seconds
Database query time	< 50 ms per query
Max tested peers simultaneously	5 peers on same WiFi
Max file size supported	10 MB

7. Conclusion

7.1 Summary of Achievements

This project successfully delivered a fully functional peer-to-peer LAN chat application with the following key achievements:

- Complete MVC architecture implemented across 12 Java source files
- Real-time TCP messaging working reliably between LAN peers
- MySQL database storing all messages, peer records, and file transfer logs
- File transfer system supporting .txt files up to 10 MB
- Discord-style browser-based web UI accessible at <http://localhost:8080> — no installation required for end users
- Auto-detection of MAC address and LAN IP with no manual configuration
- No external frameworks, no Maven, no internet dependency

7.2 Challenges Faced and Solutions

Challenge	Solution
Cross-thread browser updates: TCP messages arrive on background threads but must be pushed to the browser	Used WebSocket broadcast from the Server callback thread. ApplicationController calls <code>webServer.broadcast(json)</code> which pushes the event instantly to all connected browser tabs
MAC address detection on different OS and network adapters	Implemented a fallback chain: primary interface → scan all interfaces → hash-based fallback
File transfer reliability for large files	Implemented chunked writing (4096 byte chunks) with a loop to ensure all bytes are written
Peer discovery without a central server	Used MySQL's <code>last_seen</code> timestamp — each peer updates it on launch, others check within 3-minute threshold
No .env file for credentials	Hardcoded credentials in DBHelper.java with clear comments for easy modification

7.3 Future Enhancements

- UDP broadcast for automatic peer discovery — no database needed for finding peers
- End-to-end encryption using Java's `javax.crypto` package
- Group chat support — broadcast messages to multiple peers
- Support for additional file types (.pdf, .docx, .png)
- Online/offline status updated via heartbeat pings instead of timestamps
- Message read receipts and delivery status
- Dark mode theme toggle in the GUI

8. References

8.1 Books

1. Eckel, B. (2006). Thinking in Java (4th ed.). Prentice Hall.
2. Horstmann, C. S. (2019). Core Java Volume I — Fundamentals (11th ed.). Pearson.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

8.2 Official Documentation

4. Oracle. (2024). Java SE 21 API Documentation. <https://docs.oracle.com/en/java/javase/21/docs/api/>
5. MDN Web Docs. (2024). WebSocket API. <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
6. Oracle. (2024). Java Networking — Socket Programming. <https://docs.oracle.com/javase/tutorial/networking/sockets/>
7. MySQL. (2024). MySQL Connector/J Developer Guide. <https://dev.mysql.com/doc/connector-j/en/>
8. MySQL. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>

8.3 Online Resources

9. Baeldung. (2024). Guide to Java Sockets. <https://www.baeldung.com/a-guide-to-java-sockets>
10. Baeldung. (2024). Introduction to JDBC. <https://www.baeldung.com/java-jdbc>
11. GeeksForGeeks. (2024). MVC Design Pattern in Java. <https://www.geeksforgeeks.org/mvc-design-pattern/>
12. Stack Overflow. (2024). Java NetworkInterface MAC address detection. <https://stackoverflow.com/>